

Pemrograman Berorientasi Objek

Polimorfisme

Disusun oleh :

Tim Dosen

S1 Sistem Informasi

Fakultas Ilmu Komputer UPN "Veteran" Jakarta

Kemampuan Akhir Tahapan Belajar (Sub-CPMK)

Mahasiswa memiliki kemampuan menelaah dan menyimpulkan terkait dengan polimorfisme dan mampu mengidentifikasi jenis polimorfisme static dan dynamic serta mampu mengaitkan dengan method overloading dalam pemrograman java.

9. Polimorfisme

9.1. Konsep Polimorfisme

9.2. Studi Kasus

9.1. Konsep Polimorfisme

UNIVERSITAS PEMBANGUNAN NASIONAL
"VETERAN" JAKARTA

Makna Polymorphism

Polymorphism : polimorfisme

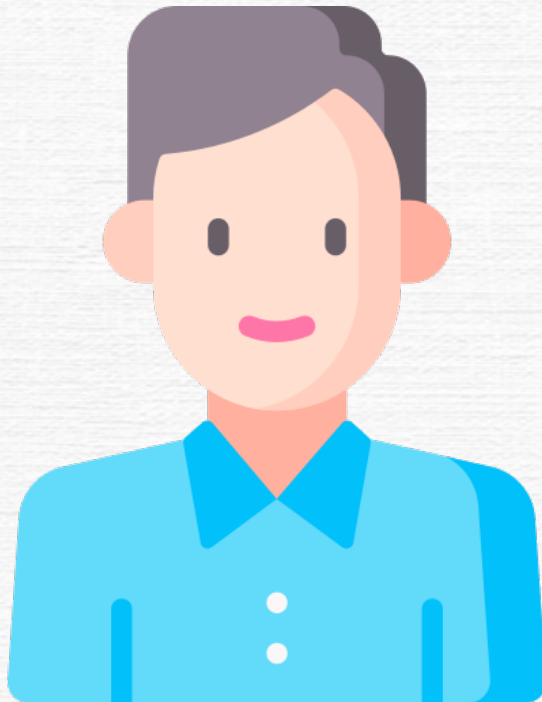
Polymorphism = **poly** (banyak) + **morphos** (bentuk) : banyak bentuk.

Polymorphism : kemampuan untuk menghandle banyak bentuk, **atau** kemampuan untuk memproses objek-objek yang **berbeda** jenis dan class melalui satu **antarmuka yang seragam**.

Key Features of Polymorphism

1. Multiple Behaviors: Metode yang sama dapat berperilaku berbeda tergantung pada objek yang memanggil metode.
2. Method Overriding: Kelas anak dapat mendefinisikan ulang metode kelas induknya.
3. Method Overloading: Kita dapat mendefinisikan beberapa metode dengan nama yang sama tetapi parameter yang berbeda.
4. Runtime Decision: Saat runtime, Java menentukan metode mana yang akan dipanggil tergantung pada kelas objek yang sebenarnya.

Ilustrasi Polimorfisme dalam Kehidupan Nyata



Bayangkan seseorang yang memainkan **peran berbeda** dalam kehidupan, seperti seorang **ayah**, seorang **suami**, dan seorang **karyawan**. Setiap peran ini menentukan perilaku orang tersebut yang berbeda, tergantung pada objek yang memanggilnya.

Role Father

```

1  // Base class Person
2  class Person {
3
4      void role() {
5          System.out.println(x:"I am a person.");
6      }
7  }
8
9  // Derived class Father
10 class Father extends Person {
11
12     // Overridden method to show
13     // the role of a father
14     @Override
15     void role() {
16         System.out.println(x:"I am a father.");
17     }
18 }
19
20 public class MainDad {
21     Run | Debug
22     public static void main(String[] args) {
23
24         // Creating a reference of type Person
25         Person p = new Father();
26
27         // Calling the role method
28         p.role();
29     }

```

Dalam contoh, Person memiliki metode role() yang mencetak pesan umum.

Kelas Father mengganti role() untuk mencetak pesan tertentu.

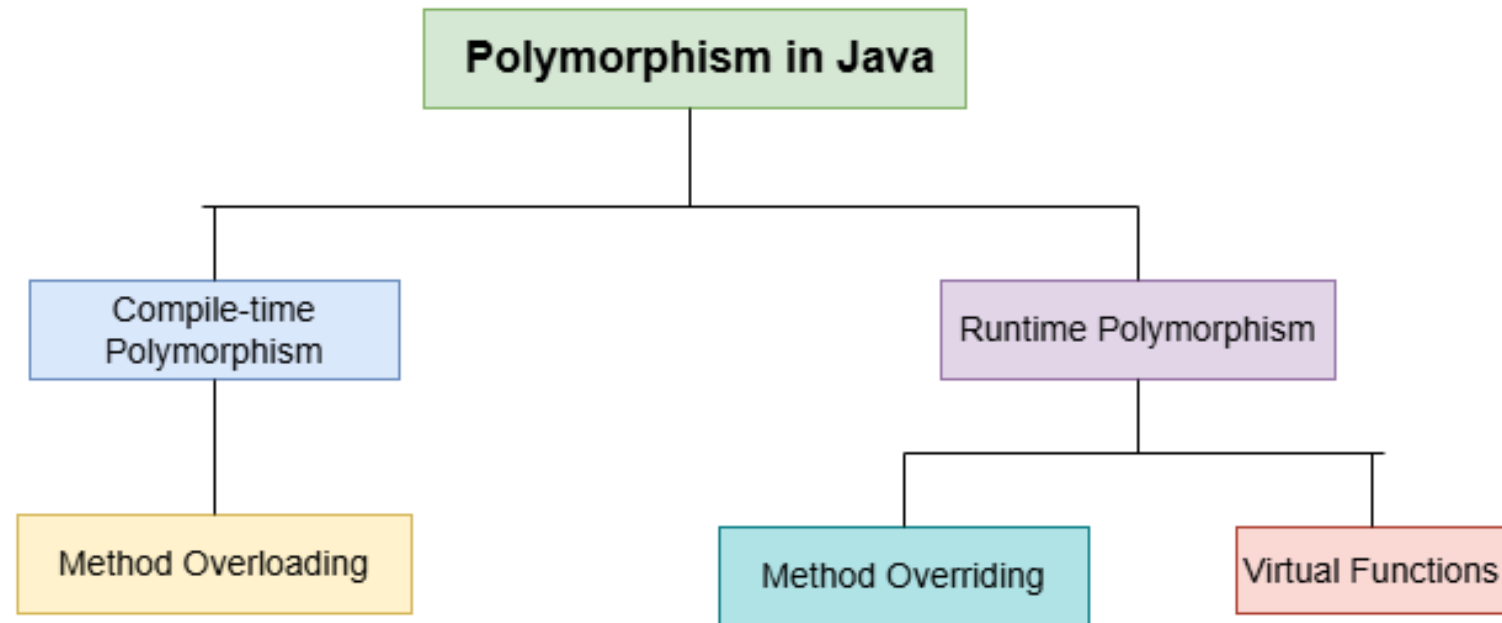
Referensi bertipe Person digunakan untuk menunjuk ke objek bertipe Father, yang menunjukkan polimorfisme saat runtime.

Metode yang diganti dalam Father dipanggil saat role() dipanggil.

Mengapa Menggunakan Polimorfisme di Java?

1. **Code Reusability** : Polimorfisme memungkinkan metode atau kelas yang sama digunakan dengan berbagai jenis objek.
2. **Flexibility** : Polimorfisme memungkinkan objek dari kelas yang berbeda diperlakukan sebagai objek dari superkelas umum, yang memberikan fleksibilitas dalam eksekusi metode dan interaksi objek.
3. **Abstraction** : Memungkinkan penggunaan kelas atau antarmuka abstrak.
4. **Dynamic Behavior** : Dapat memilih metode yang tepat untuk dipanggil saat runtime, memberikan program perilaku dinamis berdasarkan tipe objek aktual dan bukan tipe referensi, yang meningkatkan fleksibilitas.

Tipe Polimorfisme



Tipe Polimorfisme

Java memiliki 2 tipe polimorfisme:

- Polimorfisme kompilasi (static binding)

- Polimorfisme runtime (dynamic binding)

Polimorfisme statis, contoh pada method overloading.

Polimorfisme dinamis, contoh pada method overriding.

Polimorfisme Statis

Polimorfisme Waktu Kompilasi di Java juga dikenal sebagai polimorfisme statis dan juga dikenal sebagai kelebihan beban metode.

Hal ini terjadi ketika beberapa metode dalam kelas yang sama memiliki nama yang sama tetapi parameter yang berbeda.

Contoh pada method overloading.

Sewaktu kompilasi, Java tahu method mana yang akan dipanggil

Polimorfisme Statis

Aturan Method Overloading :

- a. **Nama** method harus **sama** dengan method lainnya.
- b. **Parameter** haruslah **berbeda**.
- c. Return boleh sama, juga boleh berbeda.

Polimorfisme Statis

```
package Polimorfisme1;
public class Cetak {
    // Method sama namun parameter berbeda
    // Tipe data double
    1 usage
    static double maxNumber(double a, double b) {
        if (a > b) {
            return a;
        } else {
            return b;
        }
    }
    // Method sama, namun parameter berbeda
    // Tipe data int
    1 usage
    static int maxNumber(int a, int b) {
        if (a > b) {
            return a;
        } else {
            return b;
        }
    }
    public static void main(String[] args) {
        System.out.println(maxNumber(a: 5.5, b: 7.5));
        System.out.println(maxNumber(a: 10, b: 20));
    }
}
```

Pada class Cetak memiliki 2 **method** yang sama yaitu **maxNumber()**. Tapi parameter dan tipenya berbeda, yaitu:

- static double maxNumber(double a, double b)
- static int maxNumber(int a, int b)

Yang pertama memiliki parameter dan tipe data double, sedangkan satunya lagi memiliki parameter dan tipe data int. Hal ini jelas berbeda. Inilah yang disebut Polimorfisme Statis.

Polimorfisme Dinamis

Polimorfisme dinamis identik dengan menggunakan pewarisan (inheritance), implementasi interface bahkan abstrak class.

Interface adalah class kosong yang berisi nama-nama method dan nantinya harus diimplementasikan pada class lain.

Polimorfisme Dinamis

Aturan Method Overriding

- a. Mode akses overriding method harus sama atau lebih luas daripada override method.
- b. Subclass hanya dapat dan boleh meng-override method superclass satu kali saja
- c. Tidak boleh ada lebih dari satu method yang sama pada kelas.
- d. Soal aturan hak akses, setiap subclass tidak boleh mempunyai hak akses method overriding yang ketat dibandingkan dengan hak akses method pada superclass ataupun parent class.

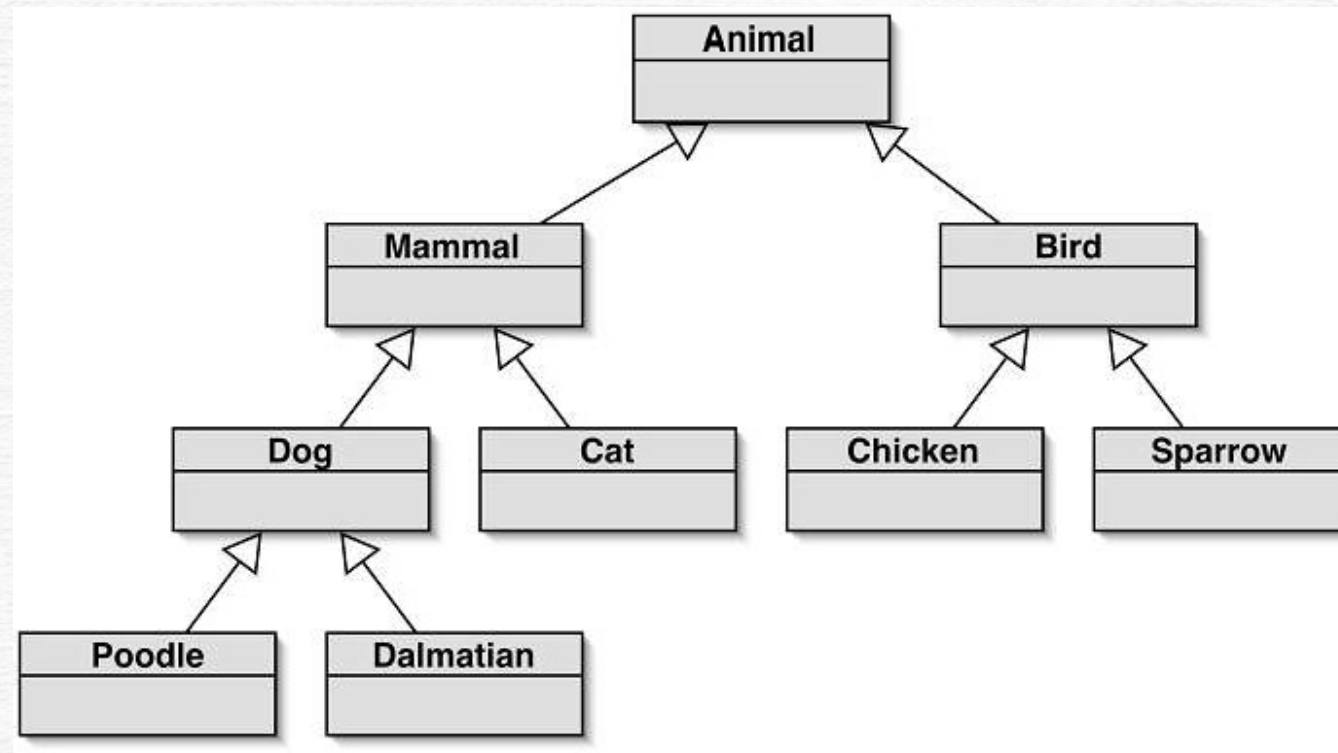
9.2. Studi Kasus

UNIVERSITAS PEMBANGUNAN NASIONAL
"VETERAN" JAKARTA

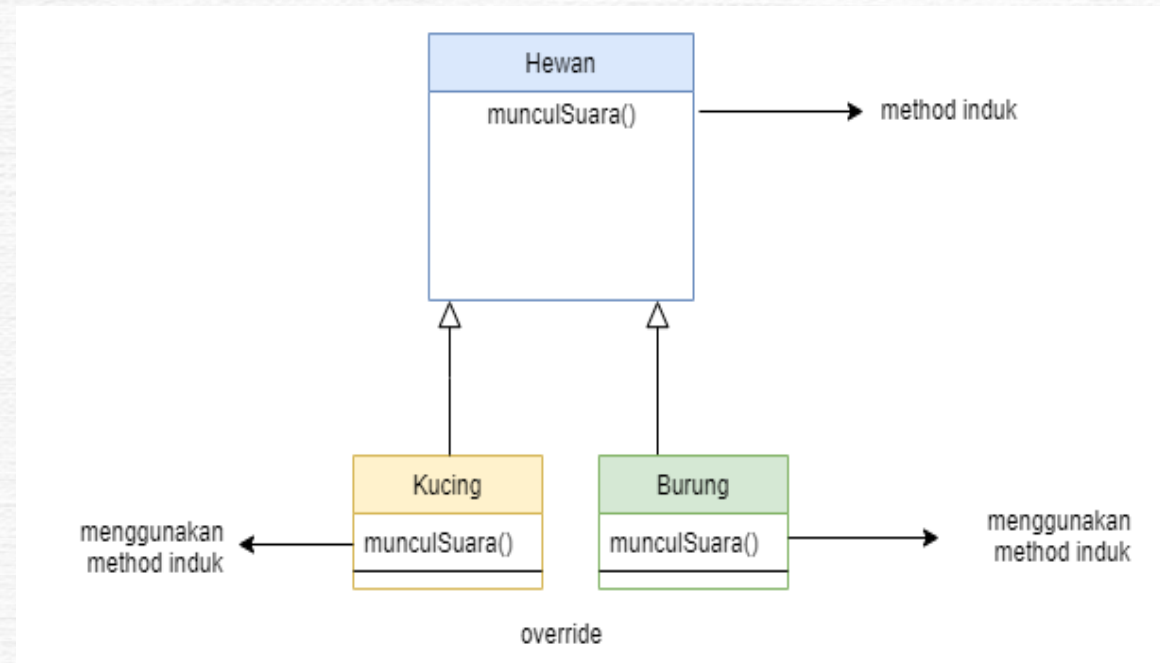
Contoh Sederhana Polimorfisme

Semua objek yang memenuhi lebih dari satu relasi is-a maka objek tersebut polymorphic.

Semua objek di Java: polymorphic



Contoh Kasus Polimorfisme



Contoh Kasus Polimorfisme

```
package Polimorfisme1;

abstract class Hewan {
    2 usages 2 implementations
    protected abstract void munculSuara();
}

1 usage
class Kucing extends Hewan {
    2 usages
    @Override
    protected void munculSuara() {
        System.out.println("Suara Kucing: Meow...meow..meow.");
    }
}

1 usage
class Burung extends Hewan {
    2 usages
    @Override
    protected void munculSuara() {
        System.out.println("Suara Burung: Cit...cit..cit.");
    }
}
```

```
package Polimorfisme1;

public class Tampil {
    public static void main(String[] args) {
        Hewan kucing = new Kucing();
        kucing.munculSuara();

        Hewan burung = new Burung();
        burung.munculSuara();
    }
}
```


Contoh Kasus Polimorfisme

```
1 class Pegawai {  
2     public void bekerja() {  
3         System.out.println(x:"Pegawai bekerja");  
4     }  
5 }
```

```
1 class Manajer extends Pegawai {  
2     @Override  
3     public void bekerja() {  
4         System.out.println(x:"Manajer mengelola tim");  
5     }  
6 }
```

```
1 class Programmer extends Pegawai {  
2     @Override  
3     public void bekerja() {  
4         System.out.println(x:"Programmer menulis kode");  
5     }  
6 }
```


Contoh Kasus Polimorfisme

```
1  public class MainEmployee {  
    Run | Debug  
2      public static void main(String[] args) {  
3          Pegawai pegawai1 = new Manajer();  
4          Pegawai pegawai2 = new Programmer();  
5  
6          pegawai1.bekerja();  
7          pegawai2.bekerja();  
8      }  
9  }
```